College0

Phase II: Design Report

Members: George Hernandez, Farida Balogun, Brian Hernandez

1. Introduction

College0 is a web-based academic management platform that supports the full lifecycle of college operations: student application and enrollment, course registration, grade management, AI-powered Q&A, review submission, and complaint tracking. The system is built on a Next.js/React frontend and a Python backend, with a RAG-based AI pipeline complemented with an LLM fallback (Gemini).

The Github repository containing all of our project's files is located at the following url:

https://github.com/bhernandez-27/CUNYzero

The four primary actors in the system are:

- Visitor – an unauthenticated user who may browse public information and submit a student application.
- Student – an authenticated user who registers for courses, views grades, submits reviews, and uses the AI assistant.
- Instructor – an authenticated user who manages course sections, posts grades, and issues warnings.
- Registrar – an authenticated administrator who processes applications, manages enrolment rules, and oversees the academic calendar.

The collaboration diagrams on the following pages capture the four core interaction flows selected for detailed design. Each diagram shows the participating objects, their stereotypes (boundary, control, entity, database), and the numbered message sequence.

Collaboration diagrams included in this section:

- UC: Register for Courses
- UC: Apply as Student + Registrar Processes Application
- UC: Join Waitlist
- UC: AI Q&A — Vector DB to LLM Fallback

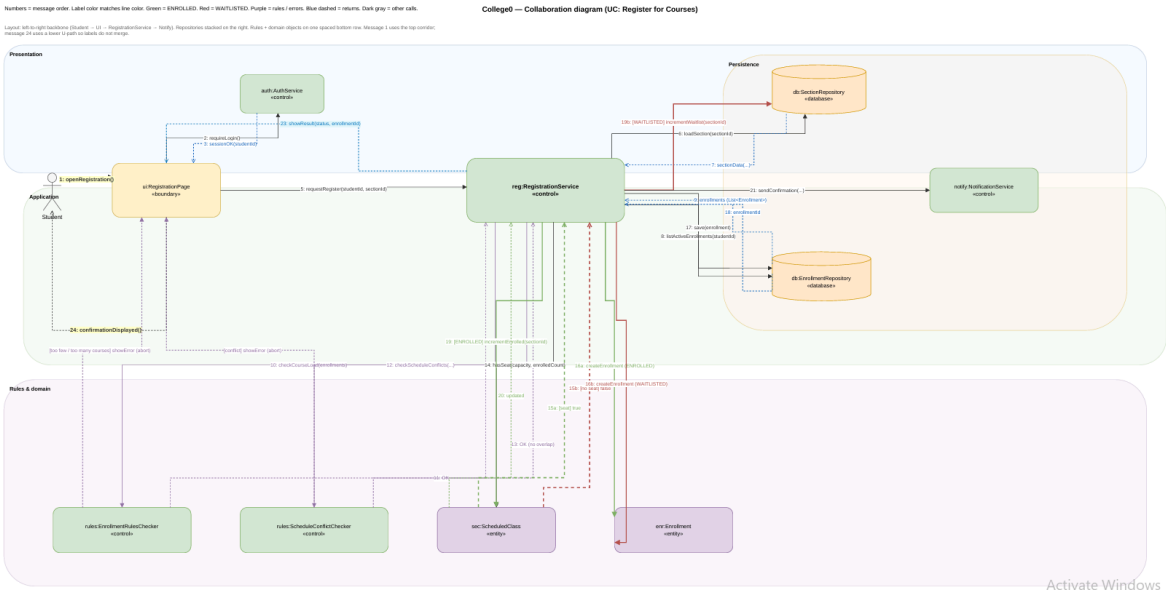


Fig 1: UC: Register for Courses

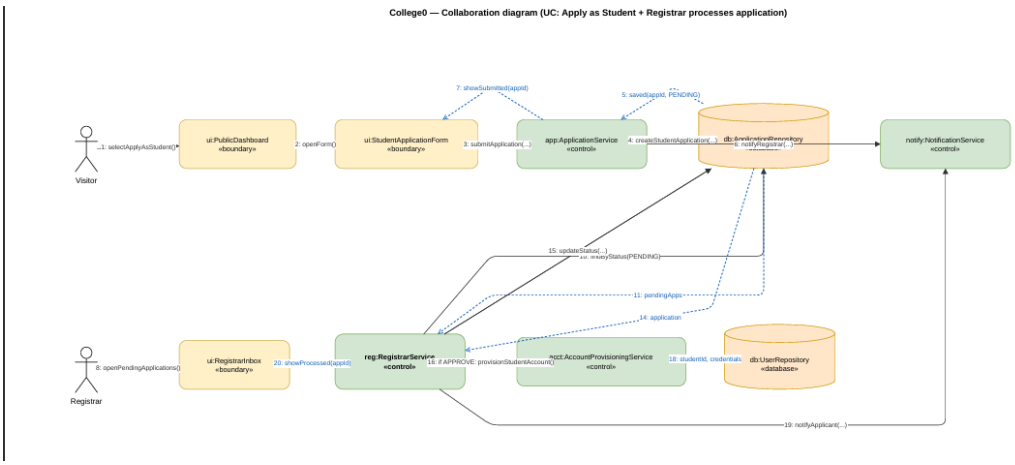


Fig 2: UC: Apply as Student + Registrar Processes Application

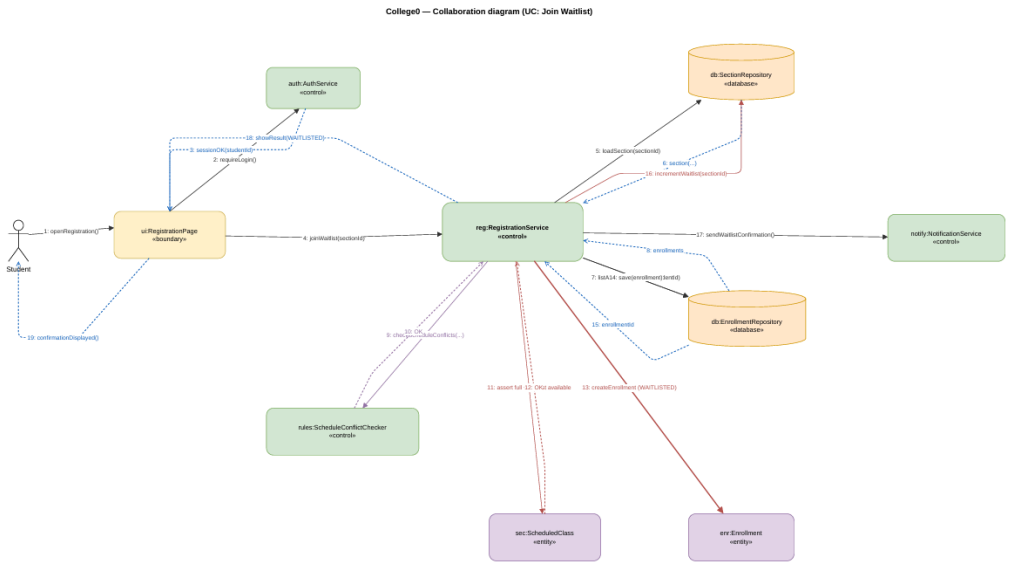


Fig 3: UC: Join Waitlist

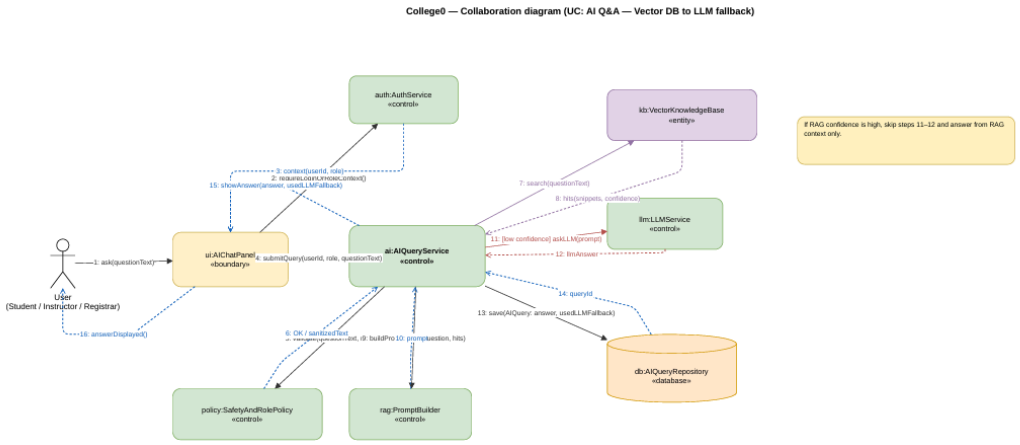


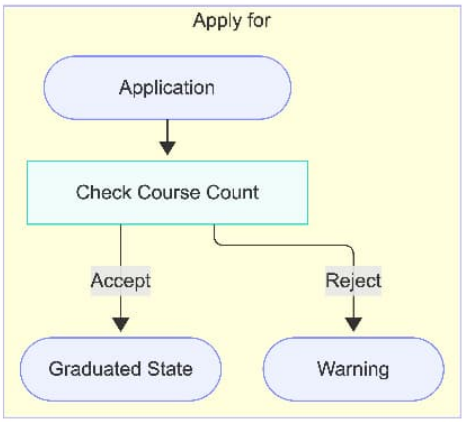
Fig 4: UC: AI Q&A — Vector DB to LLM Fallback

2. All Use Cases

This section provides normal and exceptional scenarios for each use case. Three use cases — Register for Courses, Taboo Filter (Process Review), and AI Q&A — are additionally represented with Petri nets instead of collaboration diagrams, as required.

2.1 Apply as Student

Use Case	Apply as Student
Normal Scenario	1. Visitor opens the public dashboard and selects 'Apply as Student'. 2. System renders the StudentApplicationForm. 3. Visitor fills in personal details and submits. 4. ApplicationService creates a record with status PENDING. 5. NotificationService alerts the Registrar. 6. System shows a submission confirmation with application ID.
Exceptional Scenario	• Visitor submits an incomplete form → validation error displayed, submission blocked. • A duplicate application (same email) already exists → system rejects with 'Application already on file'. • NotificationService is unavailable → application still saved; Registrar is notified via fallback log.

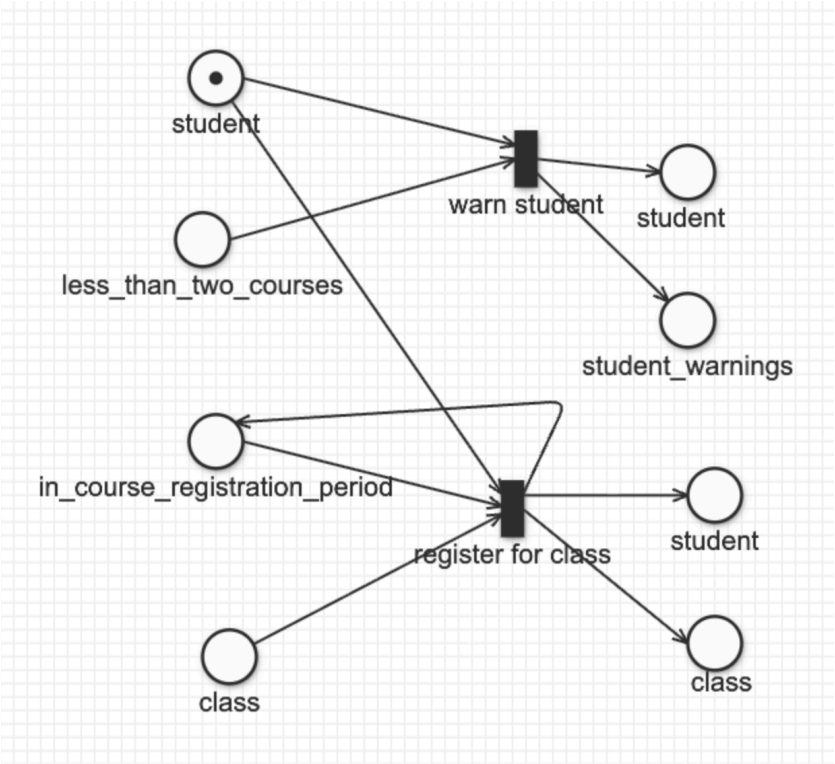


2.2 Registrar Processes Application

Use Case	Registrar Processes Application
Normal Scenario	1. Registrar opens the Registrar Inbox. 2. System lists all PENDING applications. 3. Registrar selects an application and reviews it. 4. Registrar marks it APPROVED or REJECTED. 5. If APPROVED: AccountProvisioningService creates a user account and returns credentials. 6. NotificationService emails the applicant with the decision.
Exceptional Scenario	• Application was already processed by another session → conflict error; Registrar prompted to refresh. • AccountProvisioningService fails → status rolls back to PENDING; Registrar sees an error banner. • Applicant email address is invalid → notification fails; account is still created, manual outreach flagged.

2.3 Register for Courses

Use Case	Register for Courses
Normal Scenario	1. Student navigates to the Registration page. 2. AuthService confirms session; returns studentId. 3. Student selects a section and clicks Register. 4. RegistrationService loads the section, checks course-load rules, and checks for schedule conflicts. 5. Section has a seat available → Enrollment created with status ENROLLED. 6. EnrollmentRepository saves the record; SectionRepository increments enrolled count. 7. NotificationService sends confirmation; UI shows success.
Exceptional Scenario	• Course-load limit exceeded (too many credits) → EnrollmentRulesChecker returns error; registration aborted. • Schedule conflict detected → ScheduleConflictChecker returns overlap; registration aborted. • Section is full → Enrollment created as WAITLISTED (see UC 2.4). • Database write fails mid-transaction → rollback; student sees a generic error.



2.4 Join Waitlist

Use Case	Join Waitlist
Normal Scenario	1. Student requests registration for a full section. 2. RegistrationService loads section; confirms capacity is exhausted and waitlist is open. 3. ScheduleConflictChecker passes (no time overlap). 4. EnrollmentRepository saves record with status WAITLISTED; SectionRepository increments waitlist count. 5. NotificationService sends waitlist confirmation.
Exceptional Scenario	• Waitlist is also full → system returns an error; student cannot enroll or waitlist. • Schedule conflict exists → registration aborted even for waitlist slot. • Student attempts to join the same waitlist twice → duplicate check fails; error shown.

2.5 Drop a Course

Use Case	Drop a Course

Normal Scenario	1. Student selects an active enrollment and clicks Drop. 2. RegistrationService verifies the drop deadline has not passed. 3. Enrollment status set to DROPPED; enrolled count decremented. 4. If a waitlisted student exists, system promotes them to ENROLLED and notifies them.
Exceptional Scenario	• Drop deadline has passed → system rejects with 'Drop period closed'. • Dropping below minimum credit load → EnrollmentRulesChecker blocks the action. • Promotion notification to waitlisted student fails → student is still promoted; manual follow-up logged.

2.6 View Grades

Use Case	View Grades
Normal Scenario	1. Student opens the Grade page. 2. System fetches all enrollments with final grades for the authenticated student. 3. GPA is calculated and displayed alongside individual course grades.
Exceptional Scenario	• No grades have been posted yet → page shows 'No grades available'. • GPA calculation triggers an academic standing change → SemesterStateMachine updates status (see UC 2.12).

2.7 Post Grades (Instructor)

Use Case	Post Grades
Normal Scenario	1. Instructor selects a class section they teach. 2. System lists all enrolled students. 3. Instructor enters a letter grade for each student and submits. 4. GradeRepository saves grades; academic standing re-evaluated for each student.
Exceptional Scenario	• Instructor tries to post grades after the grade submission deadline → action blocked. • Invalid grade value entered (e.g., 'Z') → validation error per student row. • Grade triggers TERMINATED status for a student → SemesterStateMachine executes termination workflow.

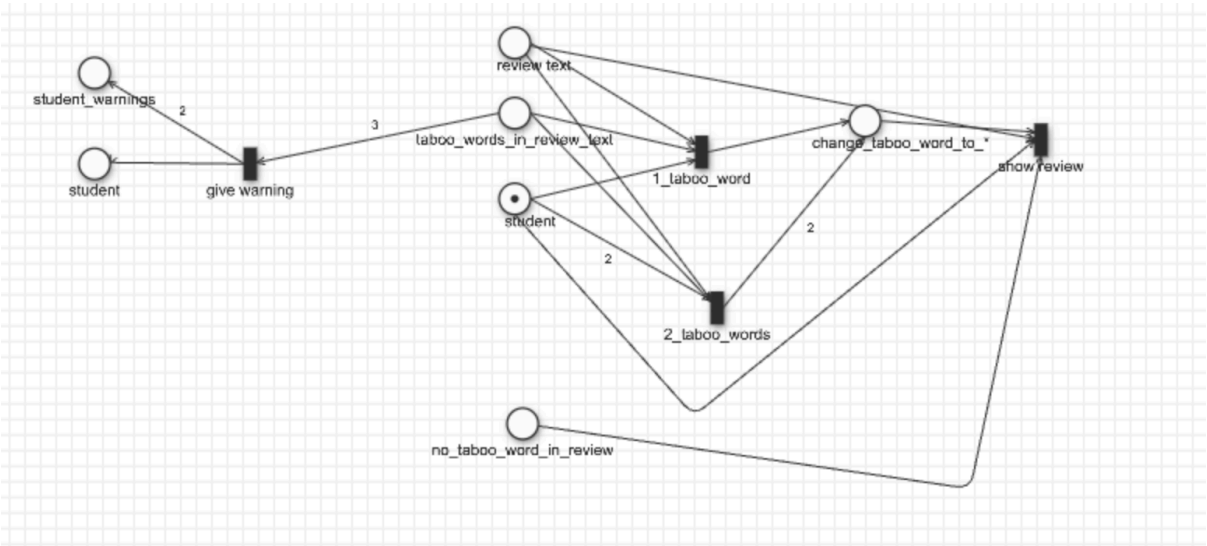
2.8 Submit a Review

Use Case	Submit a Review

Normal Scenario	1. Student opens the Review page for a completed course. 2. Student writes a text review and submits. 3. TabooFilter scans the text (see UC 2.9 / Petri Net). 4. Review is clean → saved and published immediately.
Exceptional Scenario	• 1–2 taboo words detected → words masked with '*'; warning issued; review still published. • 3+ taboo words detected → review blocked; two warnings issued; review not saved. • Student has already submitted a review for this class → system rejects with 'Review already exists'.

2.9 Taboo Filter (Process Review)

Use Case	Taboo Filter — Process Review
Normal Scenario	1. System receives review text and the active taboo word list. 2. Filter counts occurrences of taboo words. 3. Count = 0 → original text returned with 0 warnings. 4. Count 1–2 → taboo words replaced with '*'; (masked_text, 1 warning) returned. 5. Count ≥ 3 → (null, 2 warnings) returned; review blocked.
Exceptional Scenario	• Taboo word list is empty or unavailable → review passes without scanning; admin alerted. • Review text is empty → submission rejected before filter is invoked.



2.10 Issue a Warning (Instructor)

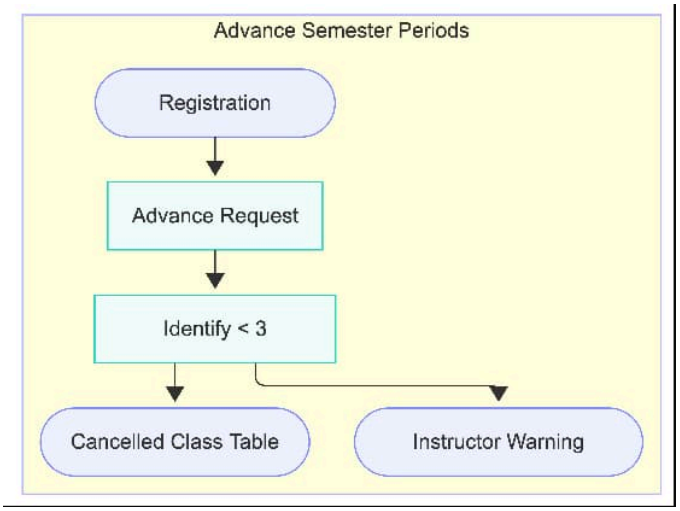
Use Case	Issue a Warning
Normal Scenario	1. Instructor identifies a student who has violated a policy. 2. Instructor opens the student's profile and issues a warning with a reason. 3. WarningRepository saves the record linked to both instructor and student. 4. NotificationService notifies the student.
Exceptional Scenario	• Instructor issues a warning for a student not in their class → authorization check blocks the action. • Student accumulates warnings triggering academic review → SemesterStateMachine transitions status.

2.11 File a Complaint

Use Case	File a Complaint
Normal Scenario	1. Student or Instructor opens the Complaint form. 2. User provides subject, description, and target (person or department). 3. ComplaintRepository saves the complaint with status OPEN. 4. Registrar is notified.
Exceptional Scenario	• Complaint is submitted with no description → validation error. • Duplicate complaint for the same incident already exists → system warns the user.

2.12 Evaluate Academic Standing (Semester State Machine)

Use Case	Evaluate Academic Standing
Normal Scenario	1. After grades are posted, SemesterStateMachine evaluates each enrolled student. 2. GPA ≥ 2.25 and not failed same course twice → status: ACTIVE. 3. GPA > 3.75 → status: HONOR_ROLL. 4. GPA 2.0–2.25 → status: WARNING_MANDATORY_INTERVIEW. 5. GPA < 2.0 OR failed same course twice → status: TERMINATED.
Exceptional Scenario	• GPA data is incomplete (missing grades) → evaluation deferred; student remains in prior status. • Student in TERMINATED status attempts future registration → system blocks with appropriate message.



2.13 AI Q&A

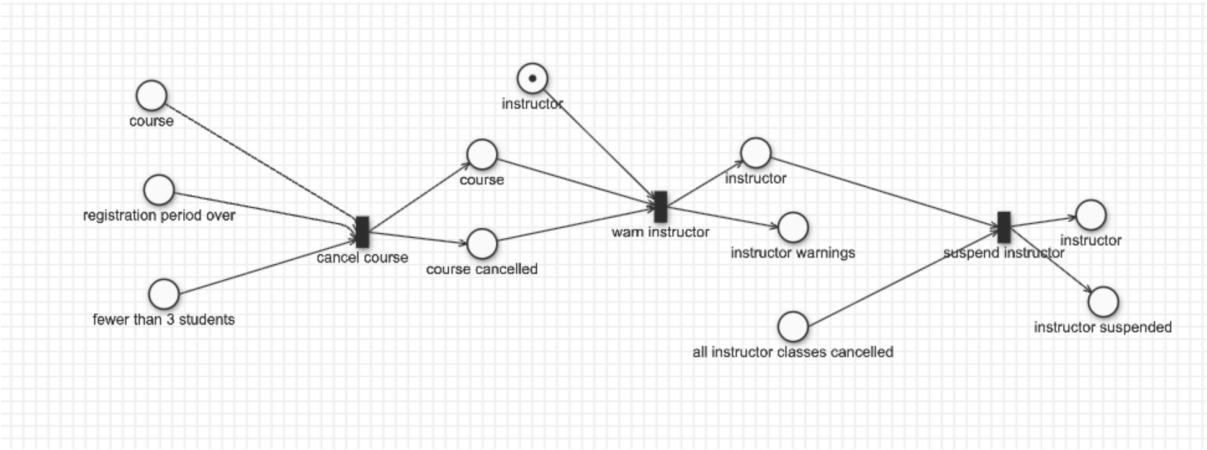
Use Case	AI Q&A
Normal Scenario	1. Authenticated user opens the AI Chat Panel and submits a question. 2. AuthService provides userId and role context. 3. SafetyAndRolePolicy sanitizes and validates the question. 4. VectorKnowledgeBase is searched; high-confidence snippets returned. 5. PromptBuilder assembles the prompt with role context and hits. 6. If confidence is high → answer served directly from RAG context. 7. If confidence is low → LLMService (Gemini) called as fallback. 8. AIQueryRepository stores the query, answer, and fallback flag. 9. UI displays the answer; halucination warning shown if LLM fallback was used.
Exceptional Scenario	• SafetyAndRolePolicy rejects the question (taboo / out of scope) → error message displayed; query not saved. • VectorKnowledgeBase returns zero results → LLM fallback always triggered. • LLM API is unavailable → system returns 'AI service temporarily unavailable'; partial RAG answer shown if available.

2.14 Manage Course Catalog (Registrar)

Use Case	Manage Course Catalog
Normal Scenario	1. Registrar opens the Course Catalog panel. 2. Registrar adds a new course or edits an existing one (name, credits, prerequisites). 3. CourseRepository saves the changes. 4. Affected class sections are updated if needed.
Exceptional Scenario	• Attempting to delete a course that has active enrollments → deletion blocked. • Invalid prerequisite chain (circular dependency) → validation error.

2.15 Cancel a Class Section

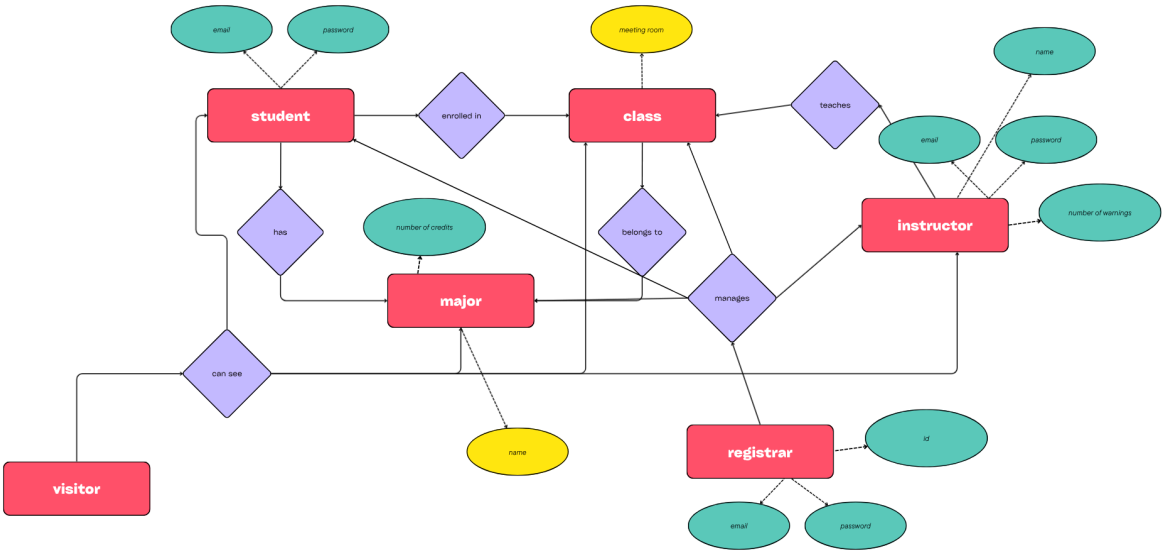
Use Case	Cancel a Class Section
Normal Scenario	1. System (or Registrar) identifies sections with fewer than 3 enrolled students. 2. Section status set to CANCELLED. 3. All enrolled/waitlisted students notified and their enrollments dropped. 4. Students receive credit back toward their load.
Exceptional Scenario	• Section is cancelled after the add/drop deadline → students receive a full refund/adjustment; Registrar notified for manual review. • Notification service fails → cancellation proceeds; manual outreach flagged.



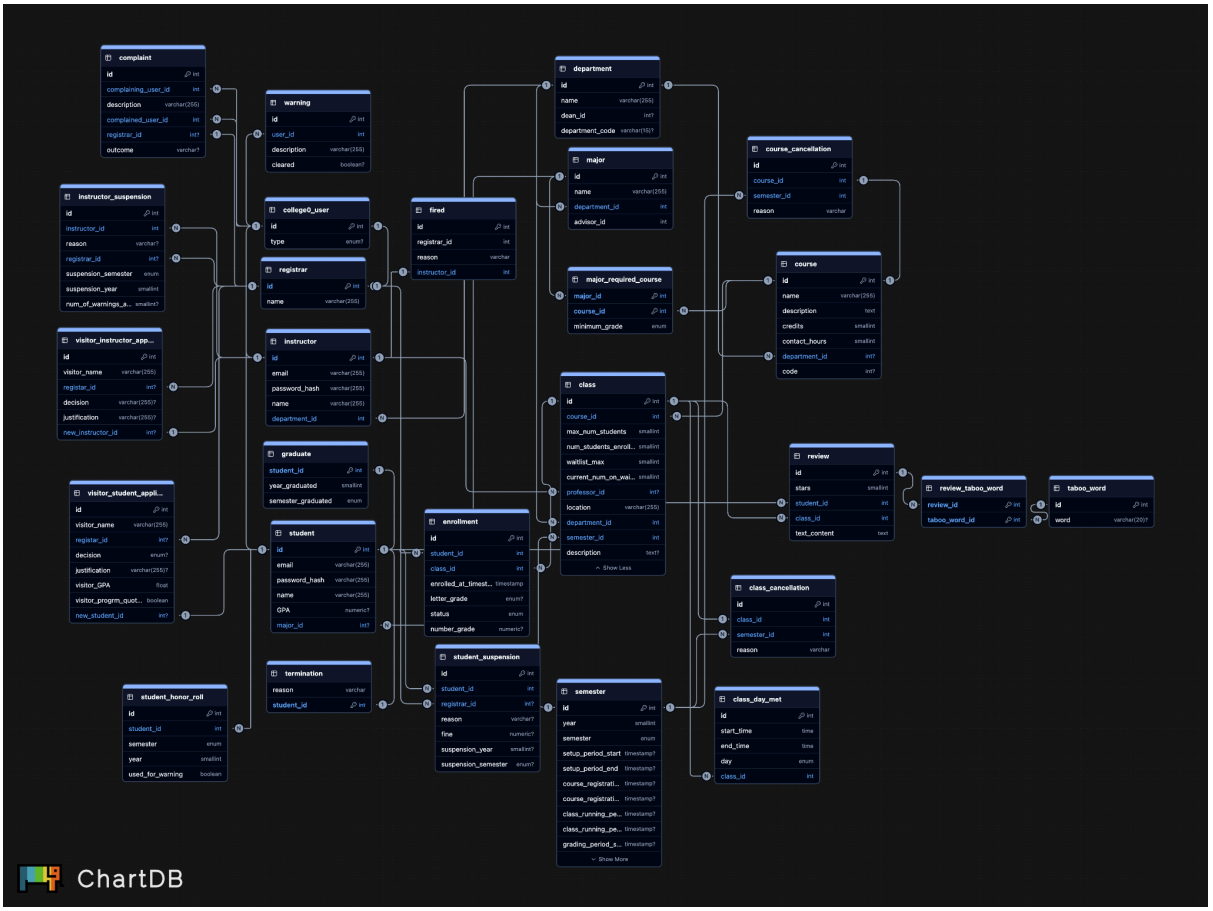
3. E-R Diagram

ER diagram - college0_user is the master identity table, and student, instructor, and registrar extend from it.

- course defines the offering, while class represents a specific section/instance of that course.
- enrollment is the bridge between student and class, showing which students are registered in which classes.
- warning captures accountability by linking an instructor who issues the warning to a student who receives it.
- review captures feedback from a student about a class.
- class_day_met stores scheduling details for when a class meets.

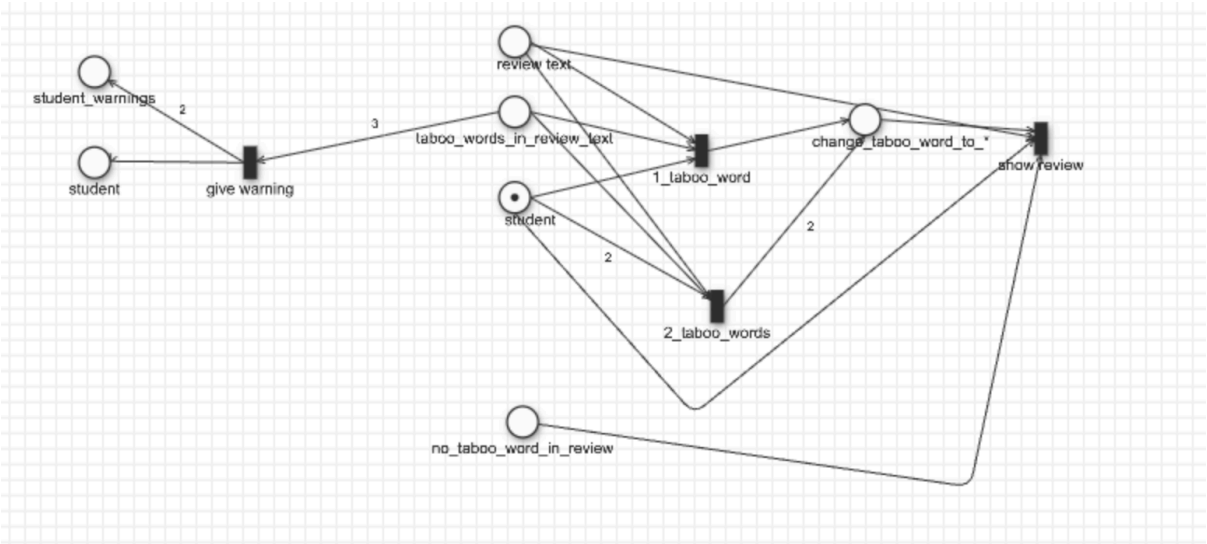


This ER diagram was based off the following Relational Diagram:

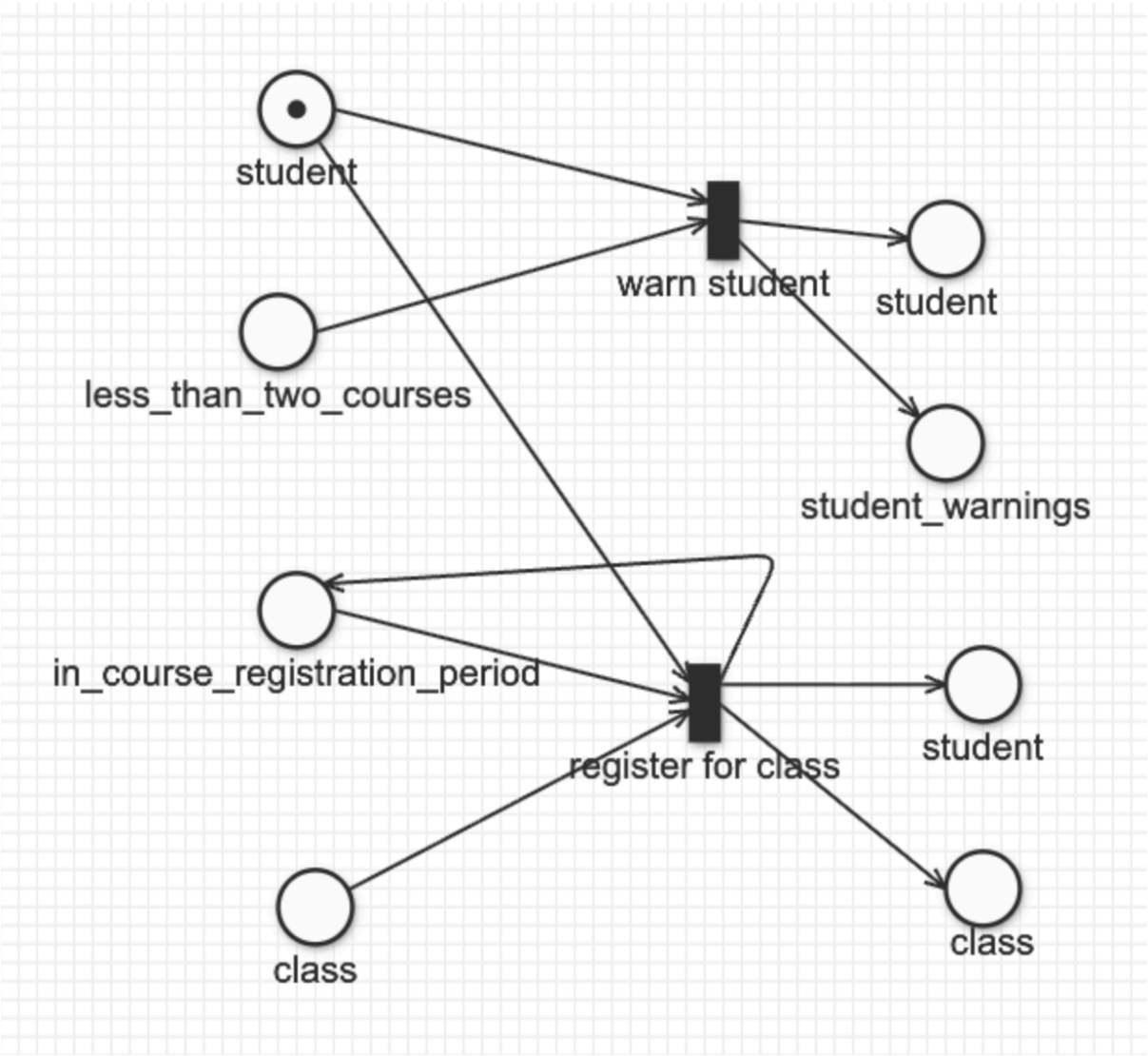


3. PetriNets

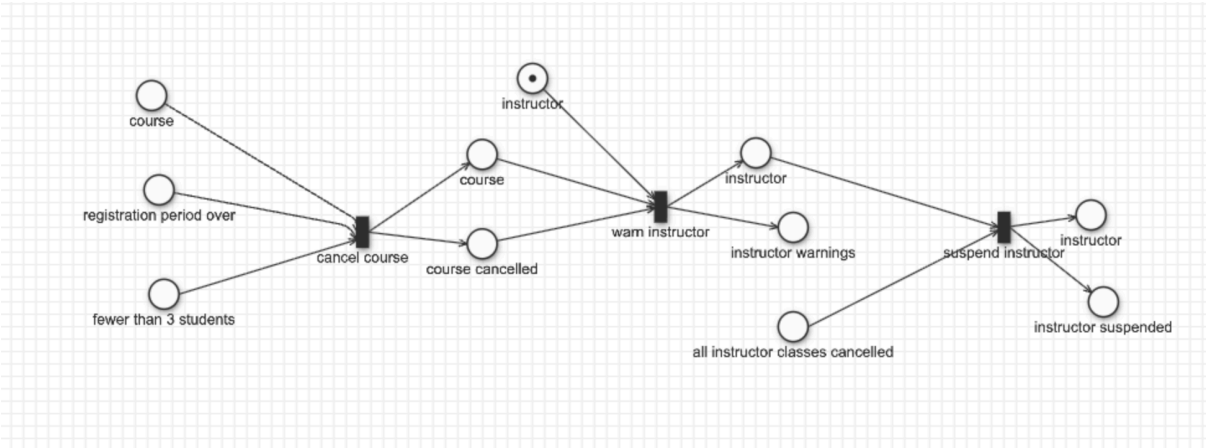
Taboo Filter: The flow starts when a user drafts a review. The system scans the text for taboo words, then routes the review into one of three outcomes: if the review is safe, it is published and the process ends; if it contains 1–2 taboo words, the text is masked, the user gets one warning, and the review is still published; if it contains 3 or more taboo words, the review is blocked, the user gets two warnings, and the process ends without publication.



Register for Course:



Cancel Course:



4. Pseudocode logic

evaluate_student_status

- Input: current_gpa (numeric), failed_same_course_twice (boolean).
- Main Function: Determines academic standing based on SRS rules.
- Logic:
    1. If current_gpa < 2.0 OR failed_same_course_twice is TRUE: Return "TERMINATED".
    2. Else If current_gpa is between 2.0 and 2.25: Return "WARNING_MANDATORY_INTERVIEW".
    3. Else If current_gpa > 3.75: Return "HONOR_ROLL".
    4. Else: Return "ACTIVE".
- Output: Status String.

process_review (Taboo Filter)

- Input: text_content (string), taboo_list (list of strings).
- Main Functionality: Analyzes and masks reviews while determining warning counts.
- Logic:
    1. Count occurrences of taboo_list words in text_content.
    2. If count $\ge$ 3: Return (null, 2 warnings).
    3. If count is 1 or 2: Mask words with "*" and Return (masked_text, 1 warning).
    4. Else: Return (original_text, 0 warnings).
- Output: Tuple (Processed Text, Warning Count).

check_overlap (Time Conflict)

- Input: start1, end1, start2, end2 (time objects)
- Main Functionality: Checks if two time intervals intersect.
- Logic:
    1. Return TRUE if (start1 < end2) AND (end1 > start2).
    2. Else: Return FALSE.
- Output: Boolean.

identify_cancelled_classes

- Input: all_classes (list of class objects).
- Main Functionality: Triggers automated cancellations for low enrollment.
- Logic:
    1. Iterate through all_classes.
    2. If num_students_enrolled < 3: Add to cancellation_list.
    3. Return cancellation_list
- Output: List of IDs to cancel.

register_for_course

- Input: student_id (int), section_id (int)
- Main Functionality: Enrolls a student in a section, or waitlists them if full, after validating all rules.
- Logic:
    - Fetch student's current enrollments and credit count.

    - Fetch section by section_id. If not found: Return error "Section not found".
    - If student credit count + section credits > max_credits: Return error "Credit load limit exceeded".
    - Fetch student's existing schedule. Call check_overlap() against new section's time slots. If TRUE: Return error "Schedule conflict detected".
    - If section.enrolled_count < section.capacity:
        - Create Enrollment with status ENROLLED.
        - Increment section.enrolled_count.
        - Call NotificationService.send_confirmation(student_id, section_id).
        - Return "ENROLLED".
    - Else if waitlist is open:
        - Create Enrollment with status WAITLISTED.
        - Increment section.waitlist_count.
        - Call NotificationService.send_waitlist_confirmation(student_id, section_id).
        - Return "WAITLISTED".
    - Else: Return error "Section full, waitlist closed".
- Output: Status string or error message.

drop_course

- Input: student_id (int), enrollment_id (int)
- Main Functionality: Drops an active enrollment and promotes a waitlisted student if one exists.
- Logic:
    - Fetch enrollment by enrollment_id. If not found or not owned by student_id: Return error "Enrollment not found".
    - If current_date > drop_deadline: Return error "Drop period closed".
    - Fetch student's current enrollments. If dropping would put credits below min_credits: Return error "Cannot drop below minimum credit load".
    - Set enrollment.status = DROPPED.
    - Decrement section.enrolled_count.
    - Call promote_from_waitlist(section_id).
    - Return "DROPPED".
- Output: Status string or error message.

promote_from_waitlist

- Input: section_id (int)
- Main Functionality: Promotes the first waitlisted student to ENROLLED when a seat opens.
- Logic:
    - Query EnrollmentRepository for the earliest WAITLISTED enrollment for section_id (ordered by created_at).
    - If none found: Return (no action taken).
    - Set enrollment.status = ENROLLED.
    - Increment section.enrolled_count.
    - Decrement section.waitlist_count.
    - Call NotificationService.send_promotion_notice(enrollment.student_id, section_id).
    - Return "PROMOTED".

- Output: Status string or void.

process_application

- Input: registrar_id (int), application_id (int), decision (string: "APPROVED" or "REJECTED")
- Main Functionality: Registrar approves or rejects a pending student application.
- Logic:
    - Fetch application by application_id. If not found: Return error "Application not found".
    - If application.status != PENDING: Return error "Application already processed".
    - If decision == "APPROVED":
        - Call AccountProvisioningService.create_account(application). If fails: Return error "Account creation failed"; do not update status.
        - Set application.status = APPROVED.
        - Call NotificationService.send_decision(application.email, "APPROVED", credentials).
    - Else if decision == "REJECTED":
        - Set application.status = REJECTED.
        - Call NotificationService.send_decision(application.email, "REJECTED").
    - Else: Return error "Invalid decision value".
    - Return decision.
- Output: Status string or error message.

query_ai

- Input: user_id (int), role (string), question (string)
- Main Functionality: Answers a question using RAG pipeline with LLM fallback if confidence is low.
- Logic:
    - Call SafetyAndRolePolicy.validate(question, role). If fails: Return error "Question rejected by safety policy".
    - Call VectorKnowledgeBase.search(question). Returns list of (snippet, confidence_score).
    - If results are non-empty AND max(confidence_score) >= CONFIDENCE_THRESHOLD:
        - Build prompt from top snippets using PromptBuilder.
        - Set answer = top snippet context. Set used_fallback = FALSE.
    - Else:
        - Call LLMService.query(question, role_context). If LLMService unavailable:
            - If partial RAG results exist: Return partial answer with warning "AI service degraded".
            - Else: Return error "AI service temporarily unavailable".
        - Set answer = LLMService response. Set used_fallback = TRUE.
    - Call AIQueryRepository.save(user_id, question, answer, used_fallback).
    - Return (answer, used_fallback).
- Output: Tuple (answer string, hallucination_warning boolean).

5. System Screens

College0 is a Next.js/React web application. Below are the major GUI screens of the system followed by a detailed prototype walkthrough of the Register for Courses functionality.

5.1 Landing Page

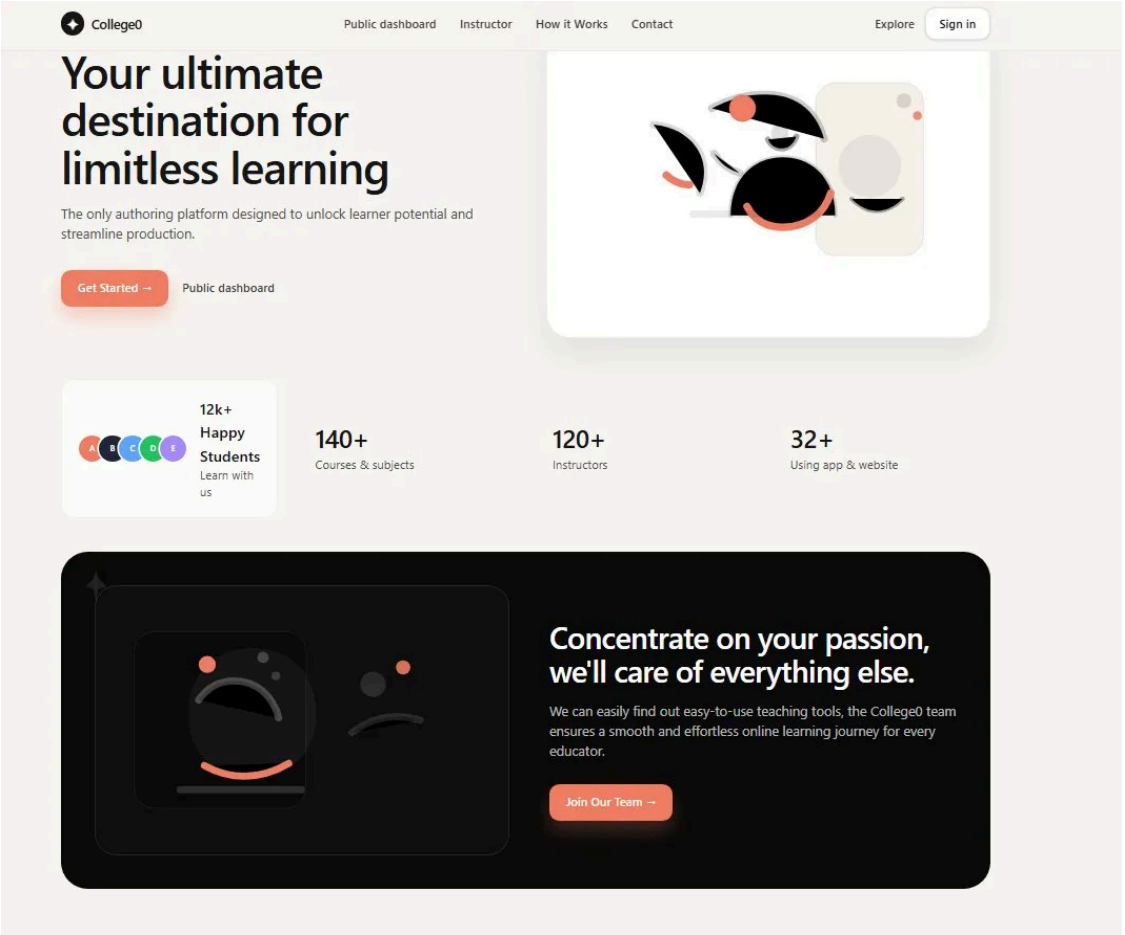


Image 1: The homepage is the first screen a Visitor sees. It features a sticky top navigation bar with links to Public Dashboard, Instructor, How it Works, and Contact, along with an Explore link and a Sign In button on the right.

5.2 Sign In Page

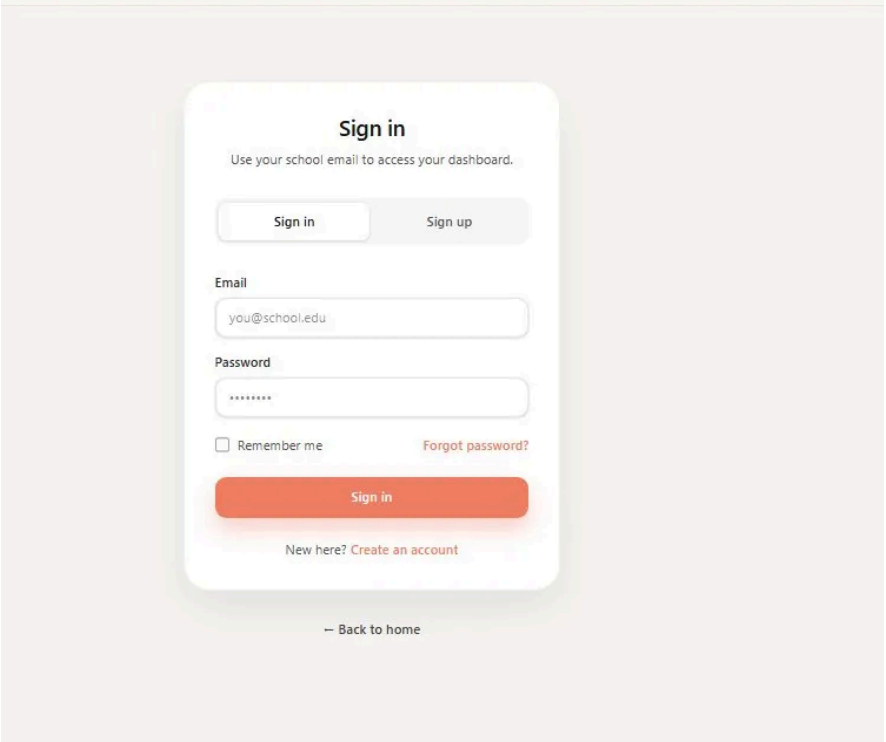


Image 2: The authentication screen presents a centered card with a Sign In / Sign Up toggle. Fields include Email (placeholder: you@school.edu) and Password. A Remember Me checkbox and Forgot Password link are provided.

5.3 Public Dashboard

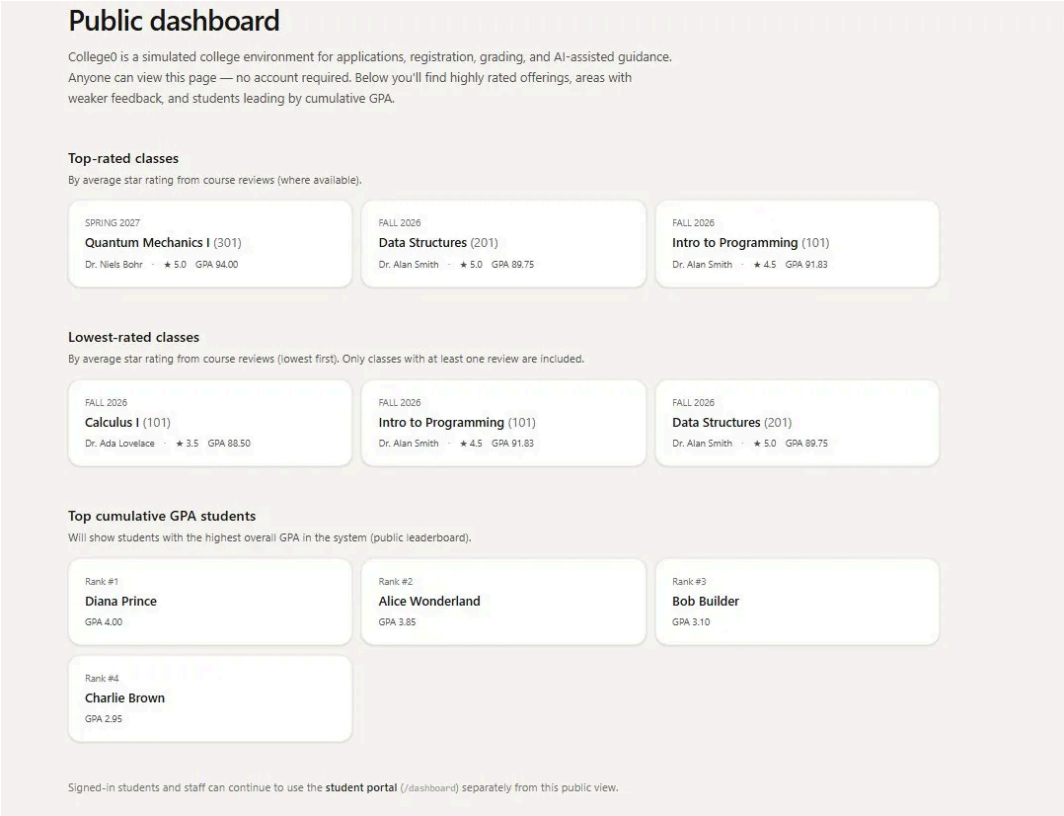


Image 3: Accessible without an account. Displays three sections: Top-rated classes (sorted by average star rating from reviews), Lowest-rated classes (only classes with at least one review), and Top cumulative GPA students (a public leaderboard ranked by overall GPA).

5.4 Prototype Register for Courses

Step	Interaction
1	Student clicks Sign In from the landing page and authenticates with their school email and password.
2	Student is redirected to the student portal (/dashboard).
3	Student clicks Registration in the left sidebar.
4	The registration page loads available sections, showing course name, section ID, instructor, time slots, and seats available.
5	Student clicks Register on a desired section.
6	A loading state is shown while the request is processed.
7a	If a seat is available: the row updates to show ENROLLED status and a confirmation toast appears.
7b	If the section is full: the row updates to show WAITLISTED status and a waitlist confirmation toast appears.

7c	**If a schedule conflict exists: an error toast appears and no enrollment is created.**
7d	**If the course load limit is exceeded: an error toast appears and the registration is blocked.**

Group Meeting Memo #1

TO: Professor Jie Wei
FROM: Farida Balogun, George Hernandez, Brian Hernandez
DATE: April 23, 2026
SUBJECT: 03/10/26 Project Meeting

On March 10, 2026 we held our first ever meeting as a team. In this meeting, we split up the workload for the CSC 332 project. We decided that George would focus on front-end design and the implementation of Gemini as a backup LLM in case the local RAG didn't provide any relevant response, Farida would work on the back-end logic, and Brian would work on the database and the RAG of the AI feature.
We also created a group chat through which we could communicate our updates without requiring us to meet together physically.

Group Meeting Memo #2

TO: Professor Jie Wei
FROM: Farida Balogun, George Hernandez, Brian Hernandez
DATE: April 23, 2026
SUBJECT: 03/19/26 Project Meeting

On March 19, 2026 we met briefly after class to discuss our progress so far after the last meeting. We originally had planned to have this meeting on 03/17/26, exactly a week after our first meeting, but scheduling conflicts meant we had to reschedule to 03/19. In this meeting, we decided that moving forward, we would communicate all updates through the group chat we had made, and not meet together physically unless absolutely required.
Since this meeting, we have not had the need to meet physically and have procedurally been implementing our updates and notifying the team of them in small increments, which has been beneficial in keeping track of the overall state of the project while allowing each teammate to stay focused on their respective modules.

Possible Concerns of Team Work Memo #2

TO: Professor Jie Wei
FROM: Farida Balogun, George Hernandez, Brian Hernandez
DATE: April 23, 2026

SUBJECT: Possible Concerns of Team Work

As of now, the most concerning possible weakness of our team is the lack of in-person communication. The loose communication style implemented is great for getting a lot of work done with each team member being able to focus 100% of their efforts on their assigned modules without having to worry about the other team members or modules. However, so much looseness may prove to be a problem later when each module needs to be connected and we potentially find that we don't understand our peer's code or find errors in the code or logic. To reduce the potential of this happening, we are each reviewing or at least skimming over each update that our team members do so that each team member has an overall idea of how the system is progressing and how the other team member's modules are being implemented. Through this incremental coding style, we minimize the chance of this concern turning into a real problem.